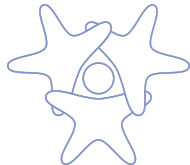




Disciplina: Engenharia de Software e Gestão de Times Ágeis

RUP e Desenvolvimento de Software

Prof. Pedro Rodolfo Gomes de Souza
prgsouza3@gmail.com



Objetivos da Aula

- Entender processos: Do cascata ao incremental, passando por espiral, reuso, RUP, Agile e DevOps.
- Conectar teoria e prática: Mostrar artefatos, papéis, fases, riscos, métricas e entregas executáveis.
- Apoiar estudo e docência: Gerar um roteiro de aula longo, progressivo e fácil de adaptar para graduação ou treinamento.



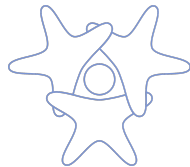
Objetivos da Aula

- Entender processos: Do cascata ao incremental, passando por espiral, reuso, RUP, Agile e DevOps.
- Conectar teoria e prática: Mostrar artefatos, papéis, fases, riscos, métricas e entregas executáveis.
- Apoiar estudo e docência: Gerar um roteiro de aula longo, progressivo e fácil de adaptar para graduação ou treinamento.



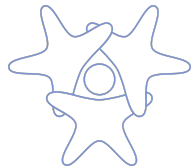
O que é um processo de software?

- Envolve entradas, saídas, responsáveis, dependências e critérios de execução.
- Ajuda a coordenar pessoas, artefatos, ferramentas e decisões.
- Não é burocracia por si só; é um mecanismo para aumentar previsibilidade e qualidade.



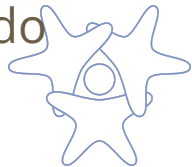
Modelo de processo

- Atividades
- O que precisa ser feito:
elicit
requisitos,
projetar,
codificar,
testar,
implantar.
- Agentes
- Papéis que executam as atividades:
analista,
arquiteto, dev,
tester, gerente,
PO.
- Artefatos
- Produtos gerados:
backlog, casos de uso,
diagramas,
código, testes,
releases,
métricas.



Processo, método e metodologia

- Processo
 - É a visão macro: fluxo de atividades, papéis, artefatos e ordem de execução.
 - Metodologia
 - Em muitos contextos, é o conjunto prático de métodos, princípios e ferramentas usado para especializar um processo em determinado ambiente organizacional.
- Método
 - É a forma sistemática de executar uma atividade específica, com passos e guias.



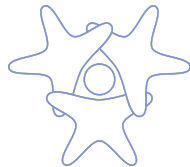
Por que processos importam?

- 1
 - 2
 - 3
 - 4
-
- Reduzem retrabalho.
 - Aumentam rastreabilidade.
 - Ajudam a gerenciar riscos.
 - Tornam a entrega mais previsível.



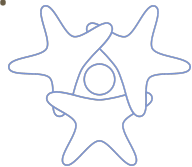
Modelo de ciclo de vida

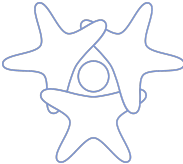
- É uma representação abstrata e simplificada de como o software nasce, evolui, é validado, implantado e mantido. O objetivo é organizar as principais atividades e os dados usados na produção e manutenção do sistema.
- Planejar
- Construir
- Evoluir



Principais modelos

- Sequenciais
- Cascata e variantes orientadas por fases.
- Evolucionários
- Programação exploratória, prototipagem e incremental.
- Iterativos
- Espiral, RUP, métodos ágeis e ciclos curtos de feedback.



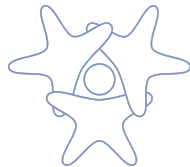


UNINASSAU
Campus Boa Viagem

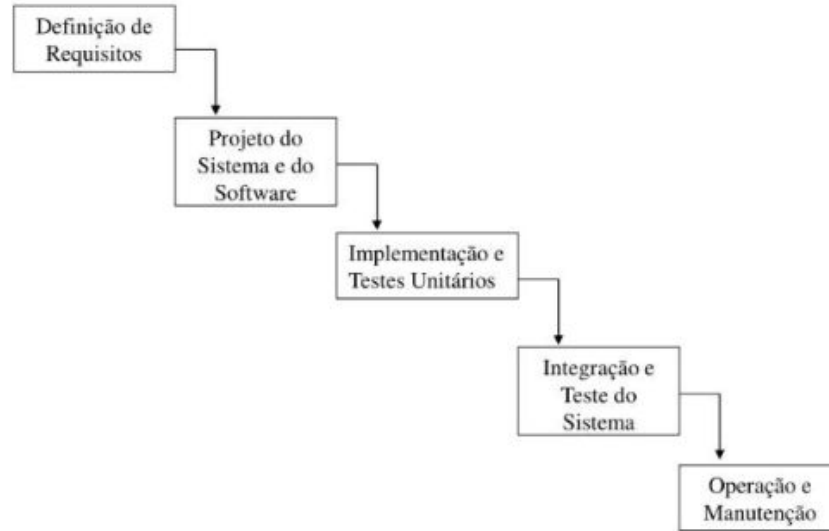
Modelos clássicos e iterativos

Cascata: ideia central

- Requisitos, projeto, implementação, testes, operação e manutenção.
- Ajuda a enxergar o projeto como uma cadeia clara de produção.
- Reage mal a mudanças tardias e pressupõe alto nível de estabilidade dos requisitos.



Cascata em cinco macroetapas



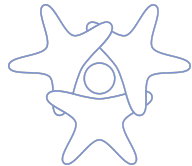
Requisitos
Definir
necessidades.

Projeto
Estruturar
solução.

Implementação
Codificar
módulos.

Integração/Teste
Verificar o
sistema.

Operação
Usar e
manter.





Vantagens do cascata

Simplicidade

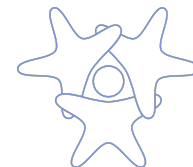
É fácil de ensinar, explicar e governar em ambientes regulados.

Documentação forte

Favorece formalização de decisões e contratos.

Marcos claros

Ajuda em auditoria, aprovação e acompanhamento gerencial.

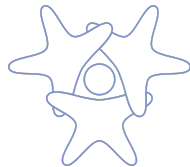


Problemas do cascata

O valor real do sistema é percebido muito tarde.

A descoberta tardia de falhas ou ambiguidades gera retrabalho elevado.

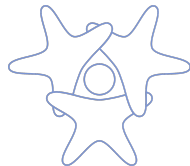
Integração, desempenho e usabilidade podem explodir apenas perto da entrega.



Programação exploratória

A primeira versão é produzida rapidamente, depois modificada sucessivamente conforme comentários dos usuários. Essa abordagem funciona quando é difícil obter uma especificação detalhada desde o início.

Rápida





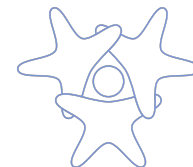
Quando a exploração faz sentido

Contextos favoráveis

Inovação, P&D, produtos com alto desconhecimento do domínio, interfaces novas e problemas mal estruturados.

Riscos associados

Arquitetura fraca, crescimento desordenado, pouca documentação e dependência de profissionais muito experientes.





Prototipagem descartável

Entender requisitos, validar interface e demonstrar viabilidade.

O protótipo não é o produto final; ele existe para aprender e depois ser reimplementado.

Não tentar colocar todo o sistema no protótipo, senão o aprendizado perde foco.



Boas perguntas antes de prototipar

O que queremos
aprender?

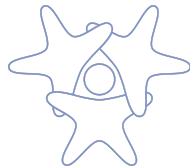
Requisitos, usabilidade, risco
técnico ou venda interna?

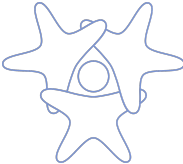
Quem valida?

Usuários, patrocinadores,
área de negócio e equipe
técnica.

O que não entra?

Escopo explícito evita
desperdício e interpretações
perigosas.





UNINASSAU
Campus Boa Viagem

Espiral, incremental e reuso



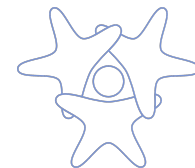
Desenvolvimento baseado em reuso

Ideia

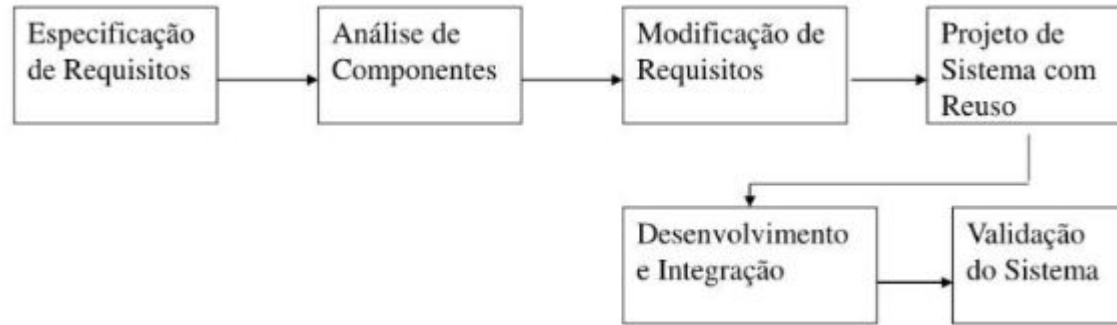
Usar componentes existentes, bibliotecas, frameworks ou sistemas COTS para acelerar a entrega.

Efeito

Muda requisitos, arquitetura e integração; desenvolver nem sempre significa construir tudo do zero.



Etapas do processo com reuso



Requisitos
Definir necessidade.

Buscar componentes
Analisar mercado e
catálogo interno.

Ajustar solução
Revisar requisitos e
arquitetura.

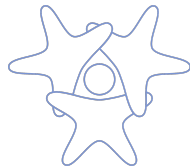
Integrar e validar
Montar, testar e
confirmar aderência.





Por que iterar?

Grandes sistemas sempre mudam durante o projeto. A iteração reconhece essa realidade e organiza o trabalho em ciclos que permitem revisar entendimento, reduzir risco e entregar partes utilizáveis do produto.

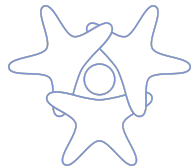
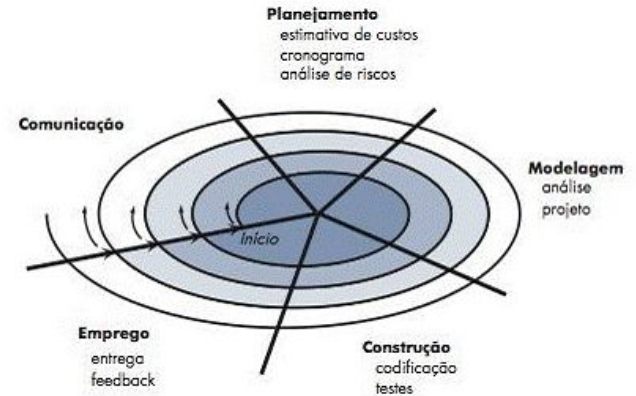


Modelo espiral

Cada volta da espiral avalia alternativas, riscos e decisões.

O foco depende do que o projeto exige naquele momento.

Planejamento, controle e validação aparecem desde cedo.





Quatro movimentos típicos da espiral

Objetivos

Definir metas e restrições.

Riscos

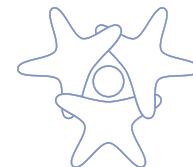
Analisar alternativas e mitigação.

Engenharia

Construir e validar versão atual.

Plano

Preparar próximo ciclo.





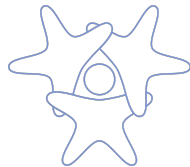
Quando usar espiral

Bom cenário

Projetos grandes, caros, críticos e com alto grau de incerteza técnica ou de negócio.

Atenção

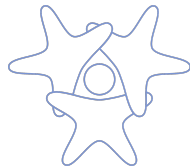
Exige maturidade em gestão, análise de risco e tomada de decisão contínua.





Desenvolvimento incremental

Em vez de entregar tudo de uma vez, o sistema é dividido em incrementos. As funcionalidades de maior prioridade entram primeiro, e cada incremento produz parte do valor esperado pelo usuário.



Fluxo típico do incremental

1. Esboçar requisitos

Definir o escopo global.

2. Priorizar e distribuir

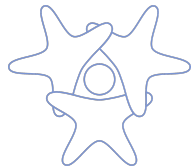
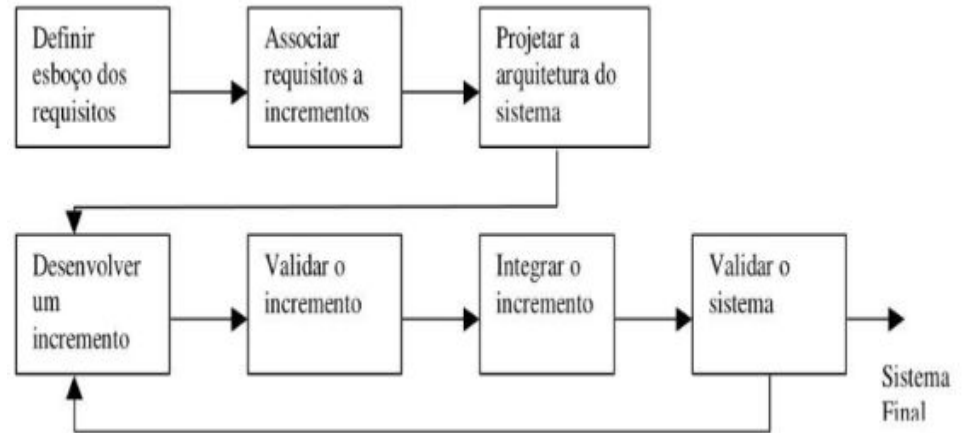
Associar requisitos a incrementos.

3. Projetar arquitetura

Criar base estável.

4. Desenvolver e validar

Entregar incremento executável.





Benefícios do incremental

Valor cedo

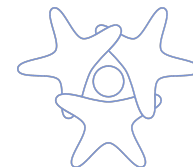
Usuário vê resultados antes
do fim do projeto.

Risco diluído

Problemas aparecem em
lotes menores e mais
controláveis.

Aprendizado contínuo

Cada entrega melhora a
próxima iteração.





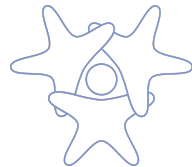
UNINASSAU
Campus Boa Viagem

Visão geral do RUP



O que é o RUP?

O nome significa Rational Unified Process. Na prática, ele combina processo, métodos, guias e uso de UML para organizar o desenvolvimento.



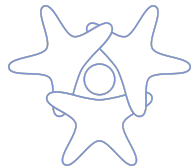
RUP como framework

Ideia principal

Mais do que um roteiro fixo, o RUP foi concebido como um framework configurável para gerar processos adaptados ao contexto do projeto.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Características centrais

Definição

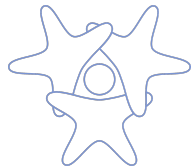
O RUP é iterativo e incremental, guiado por casos de uso e centrado na arquitetura do sistema.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



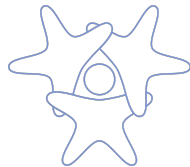
Quatro fases do ciclo



Concepção, elaboração, construção e transição compõem o ciclo de vida do RUP.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

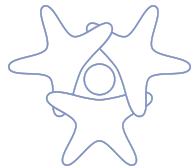
É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Concepção

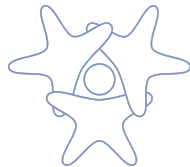
Define escopo, visão, viabilidade, principais requisitos, estimativas iniciais e riscos principais.





Elaboração

Detalha requisitos prioritários e estabiliza a arquitetura executável do sistema.





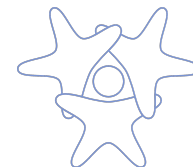
Construção

Ideia principal

Implementa a maior parte das funcionalidades, amplia qualidade e prepara o produto para uso real.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





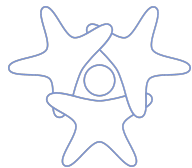
Transição

Definição

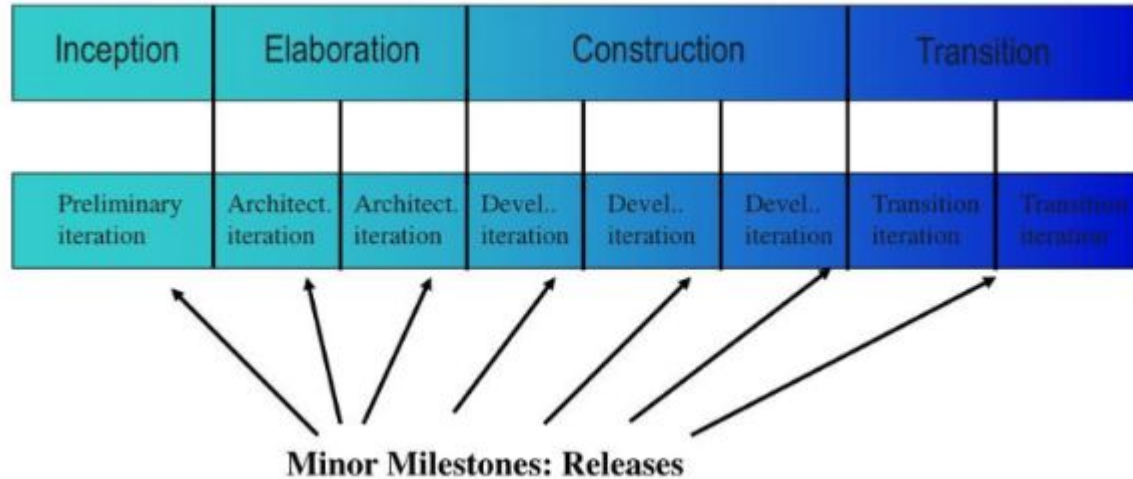
Implanta o sistema,
treina usuários,
corrige defeitos
finais e faz a
passagem ao
ambiente
operacional.

Valor

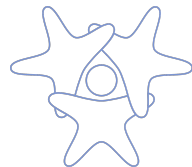
Ajuda a reduzir
incerteza e
melhorar qualidade
das decisões.



Milestones no RUP

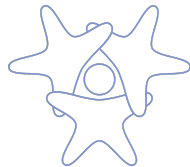


Cada fase termina com um marco importante para decidir se o projeto avança com segurança.



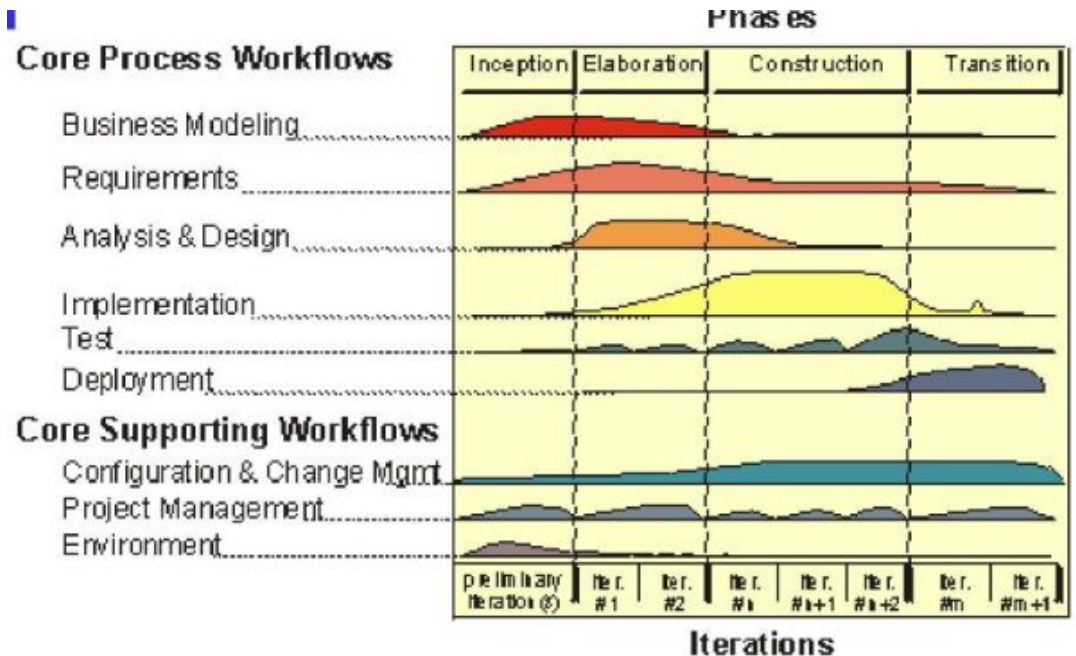
Iterações dentro das fases

Toda fase possui uma ou mais iterações planejadas, cada uma com objetivos, atividades e critérios de sucesso.



Entregas executáveis

Em geral, cada iteração produz uma versão executável ou demonstrável do sistema.





RUP e casos de uso

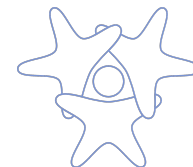
Ideia principal

Casos de uso orientam requisitos, planejamento de iterações, projeto, integração e testes.

RUP e Arquitetura

Definição

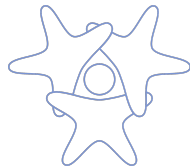
A arquitetura é definida cedo e usada como base para crescimento controlado do sistema.





Vantagem gerencial do RUP

Permite visibilidade maior do andamento e checkpoints formais entre fases e iterações.



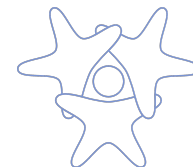
RUP x Outras metodologias

RUP vs Cascata

As fases existem, mas dentro delas há iteração, feedback e validação progressiva, não linearidade absoluta.

RUP vs AGILE

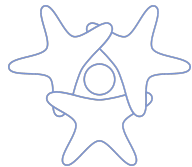
RUP tende a ser mais prescritivo e documentado;
Agile costuma priorizar leveza, adaptabilidade e comunicação direta.



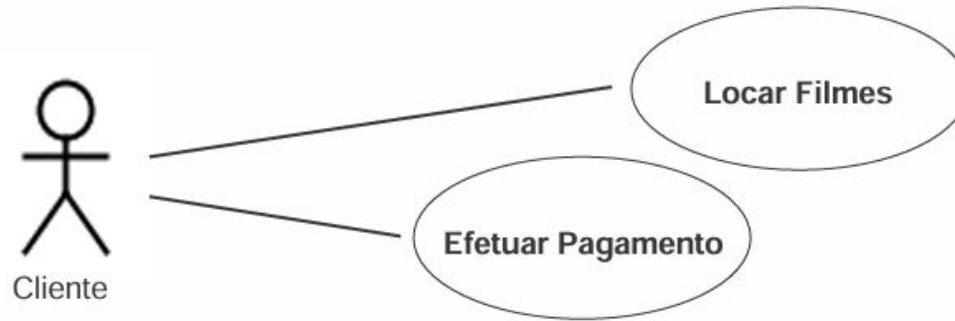


RUP e disciplina

O famoso dilema é equilibrar agilidade e disciplina em vez de tratar os dois como opostos absolutos.



Casos de uso: conceito

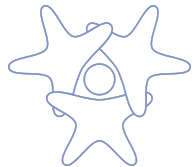


Descrevem interações entre atores e sistema para atingir objetivos observáveis do negócio.

Cada cenário importante pode originar testes de aceitação e casos de teste detalhados.

Contém Nome, objetivo,
atores, fluxo principal, fluxos
alternativos, pré-condições e
pós-condições.

No RUP, eles ajudam a decidir ordem das iterações e prioridades de implementação.

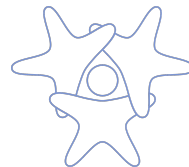


Arquitetura de software

É a visão de alto nível do sistema, seus subsistemas, componentes e relacionamentos.

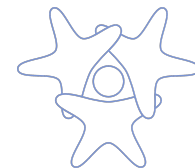
Ela organiza o time, facilita reuso e reduz risco técnico em sistemas complexos.

Decisões arquiteturais afetam desempenho, segurança, disponibilidade, escalabilidade e manutenção.



O que são requisitos?

- Requisitos
 - Descrições dos serviços prestados pelo sistema e das restrições a sua operação
- Engenharia de Requisitos
 - Processo de descobrir, analisar, documentar e verificar requisitos



Visões da Especificação

- É interessante gerar diferentes visões da especificação de sistema
 - Comunicação com diferentes tipos de interessados (stakeholders)



Requisitos do Usuário



Gerentes do cliente
Usuários finais
Engenheiros do cliente
Fornecedores

Requisitos de Sistema

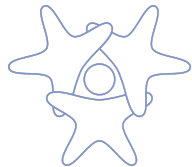


Analistas do sistema
Arquitetos de software
Desenvolvedores



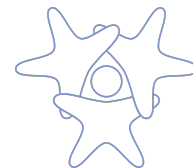
Requisitos Funcionais

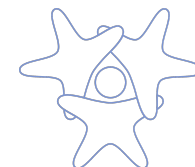
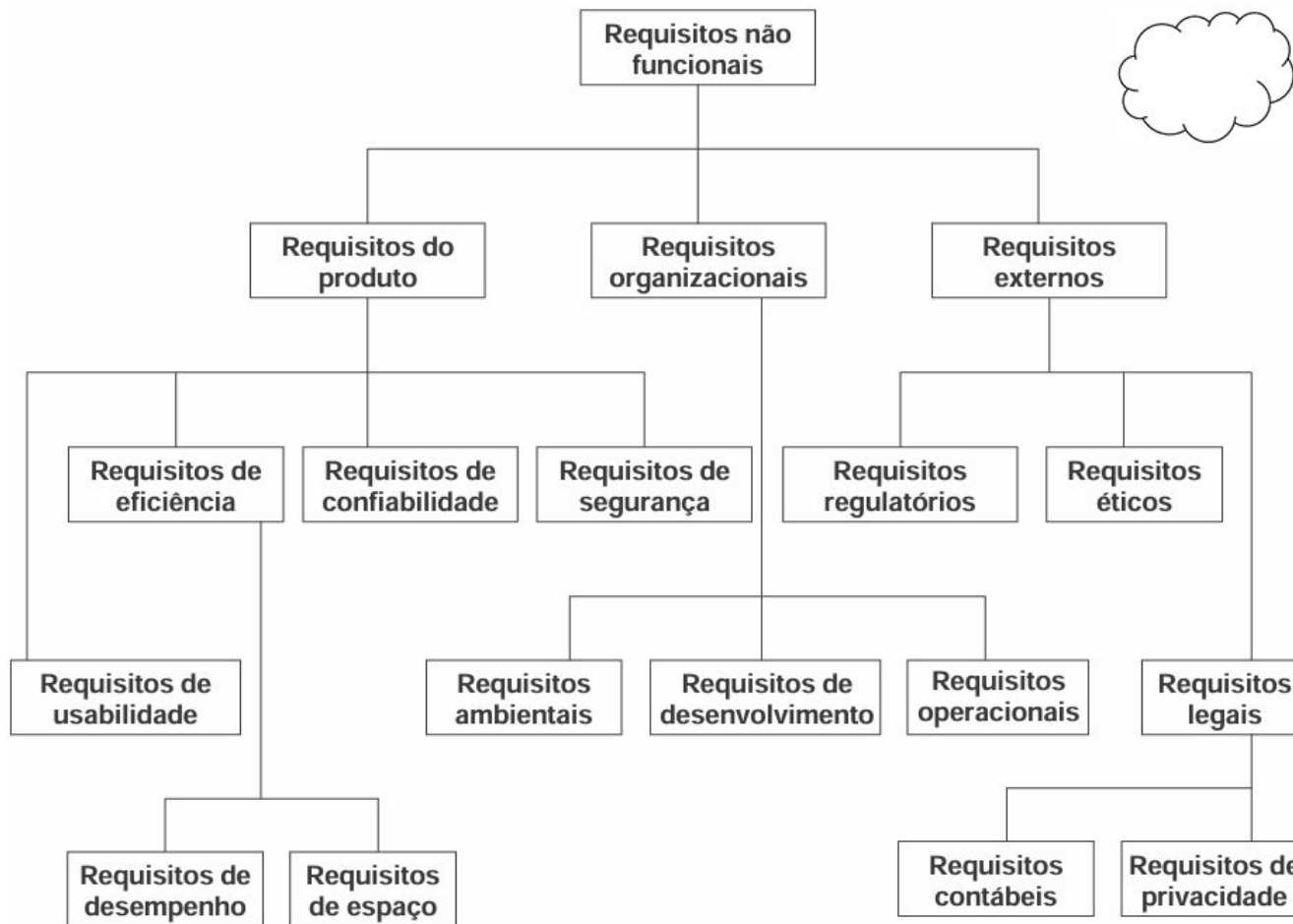
- Declarações de serviços que o sistema deve fornecer
- Documenta
 - como o sistema deve reagir a entradas específicas
 - como deve se comportar em determinadas situações
 - o que o sistema não deve fazer



Requisitos Não-Funcionais (RNF)

- Restrições sobre os serviços ou funções do sistema
 - Exemplos: confiabilidade, desempenho, espaço em disco
- Podem ser do sistema todo ou de partes do sistema
- Requisitos não-funcionais podem ser mais críticos que requisitos funcionais
 - Se não satisfaz, o sistema é inútil







UML: visão geral

A UML é uma linguagem de modelagem amplamente usada para representar estrutura e comportamento de sistemas.

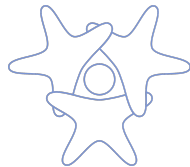


Diagrama de casos de uso

Mostra atores, casos de uso e relações de forma resumida para comunicação de escopo.

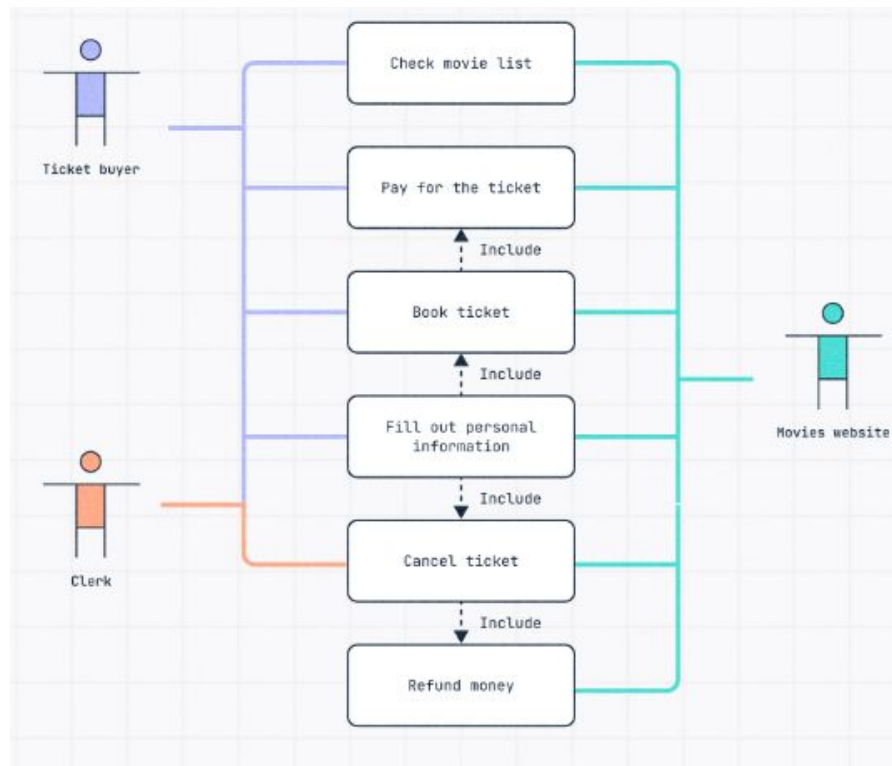


Diagrama de classes

Mostra classes, atributos, operações e relacionamentos como associação e herança.

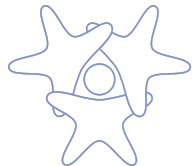
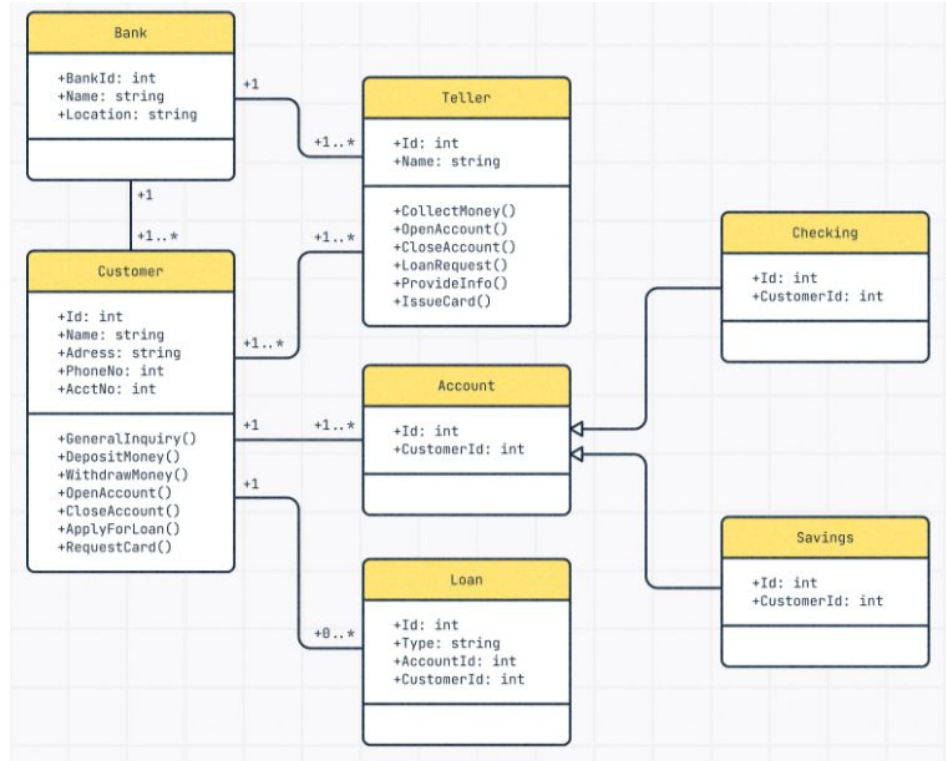
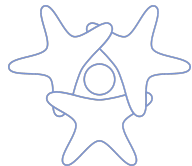
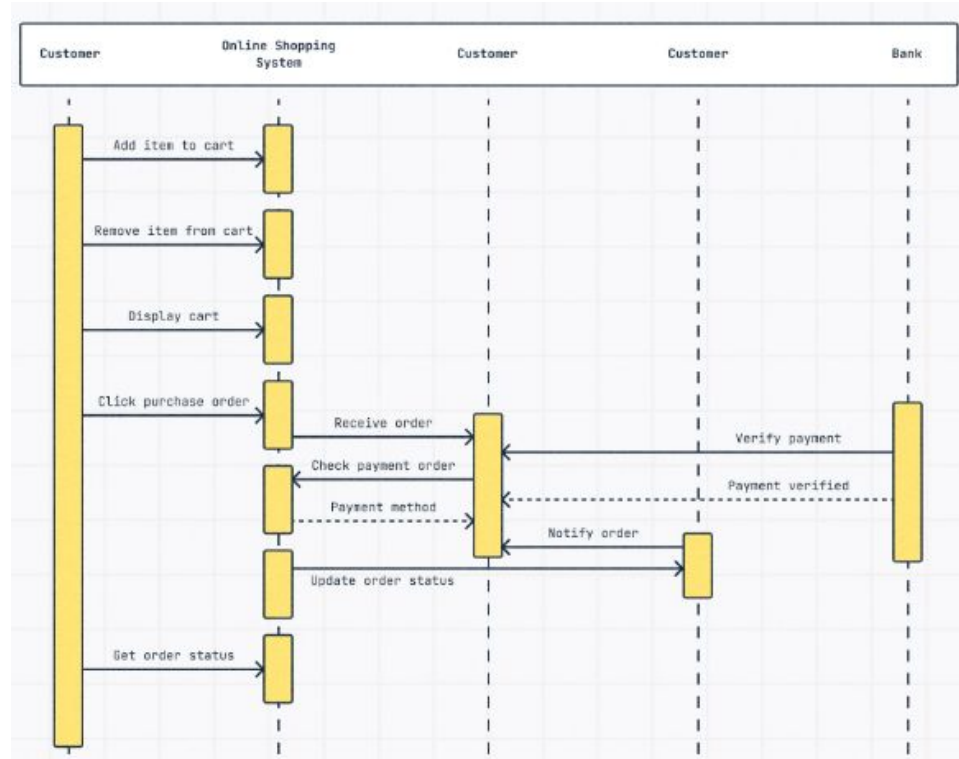


Diagrama de sequência

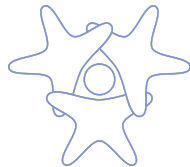
Mostra a ordem das mensagens entre objetos para realizar um cenário específico.





Modelagem não é fim em si

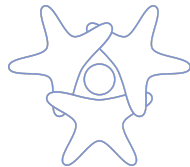
Diagramas devem esclarecer decisões e comunicação, não virar burocracia ornamental.





Análise e projeto

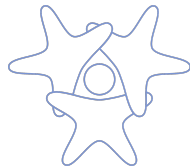
Na análise, entendemos o problema; no projeto, definimos a solução técnica.





Do requisito ao código

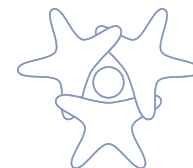
Uma cadeia saudável liga necessidade de negócio,
modelo, arquitetura, implementação e teste.



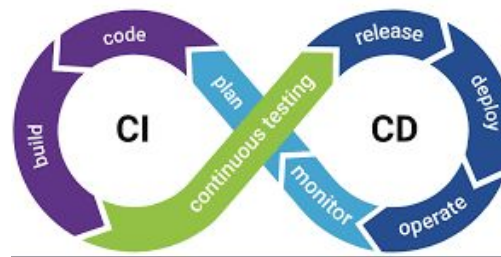


Implementação

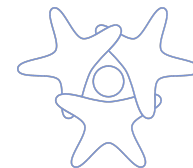
Codificar envolve padrões, revisão, automação, integração contínua e disciplina técnica.



Integração contínua



Mudanças frequentes no código são integradas e verificadas automaticamente para reduzir surpresas.

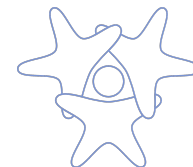




Qualidade de código

Legibilidade, testabilidade, simplicidade e consistência importam tanto quanto funcionalidade.

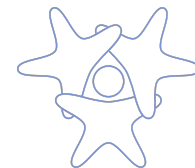
O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.





Testes unitários

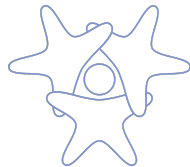
Validam pequenas unidades de código e ajudam a detectar regressões cedo.





Testes de integração

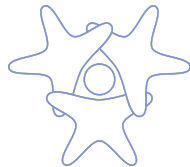
Avaliam interfaces entre módulos, serviços, banco de dados e sistemas externos.





Testes de sistema

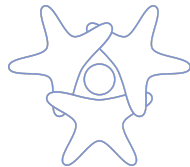
Verificam comportamento do produto integrado em relação aos requisitos definidos.

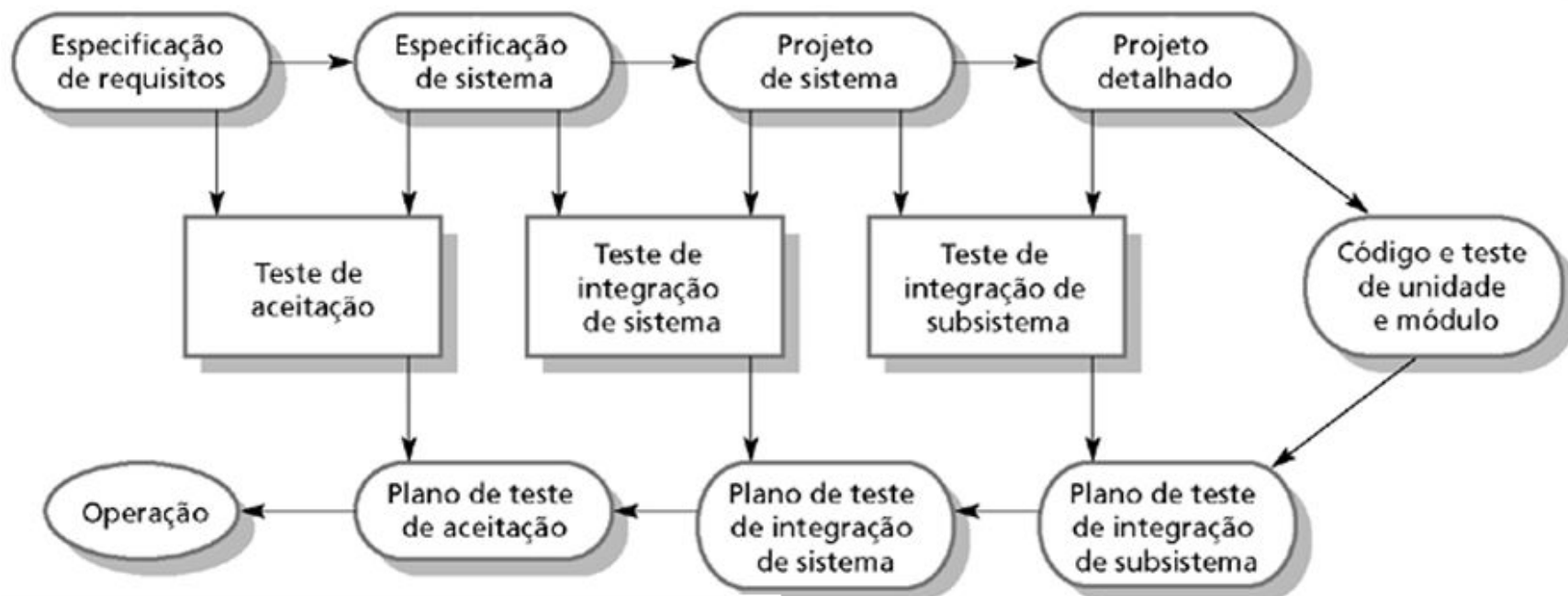




Testes de aceitação

Confirmam se o software atende à expectativa de negócio e do usuário final.

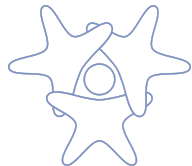




Gerência de configuração



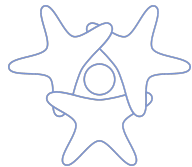
Controla versões de artefatos e garante integridade do produto ao longo do tempo.





Gestão de mudanças

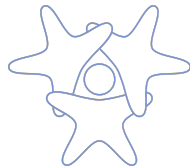
Toda mudança relevante precisa ser registrada, analisada, priorizada e acompanhada.





Gestão de projeto

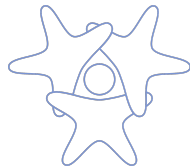
Escopo, prazo, custo, qualidade, riscos, comunicação e pessoas compõem o núcleo da gestão.





Riscos em software

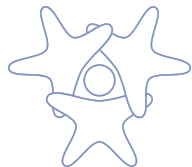
Podem ser técnicos, organizacionais, financeiros, de cronograma, legais ou humanos.



Análise de risco



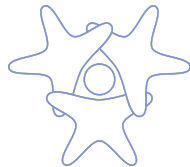
Probabilidade e impacto ajudam a priorizar mitigação, contingência e monitoramento.





Métricas de software

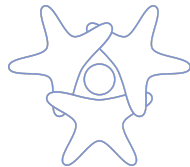
Velocidade, lead time, cobertura, defeitos e satisfação ajudam a enxergar desempenho.





Cuidado com métricas

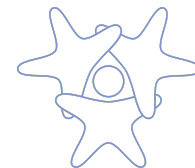
Métrica ruim gera comportamento ruim; medir só produtividade pode incentivar dívida técnica.

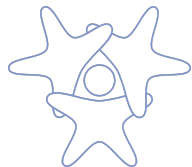
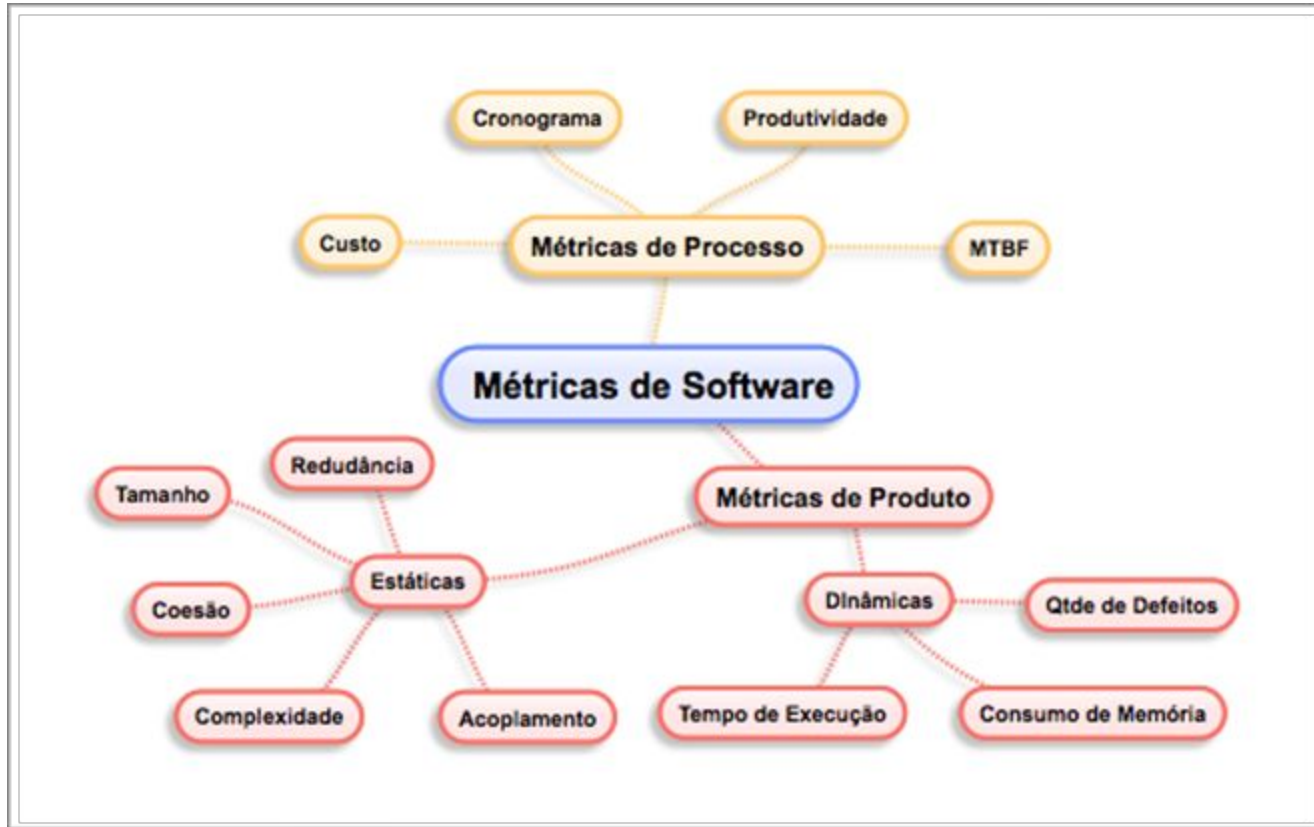




Documentação útil

Boa documentação reduz dependência de memória oral e acelera onboarding e manutenção.

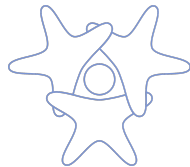






Agile: princípios gerais

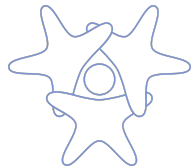
Pequenos ciclos, colaboração, adaptação e foco em software funcionando.





Scrum em resumo

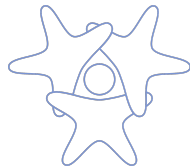
Papéis, eventos e artefatos organizam trabalho iterativo em sprints com inspeção e adaptação.





XP em resumo

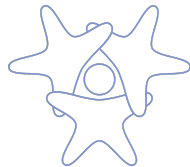
Valoriza práticas de engenharia como TDD, refatoração, integração contínua e programação em par.



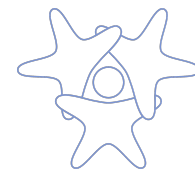
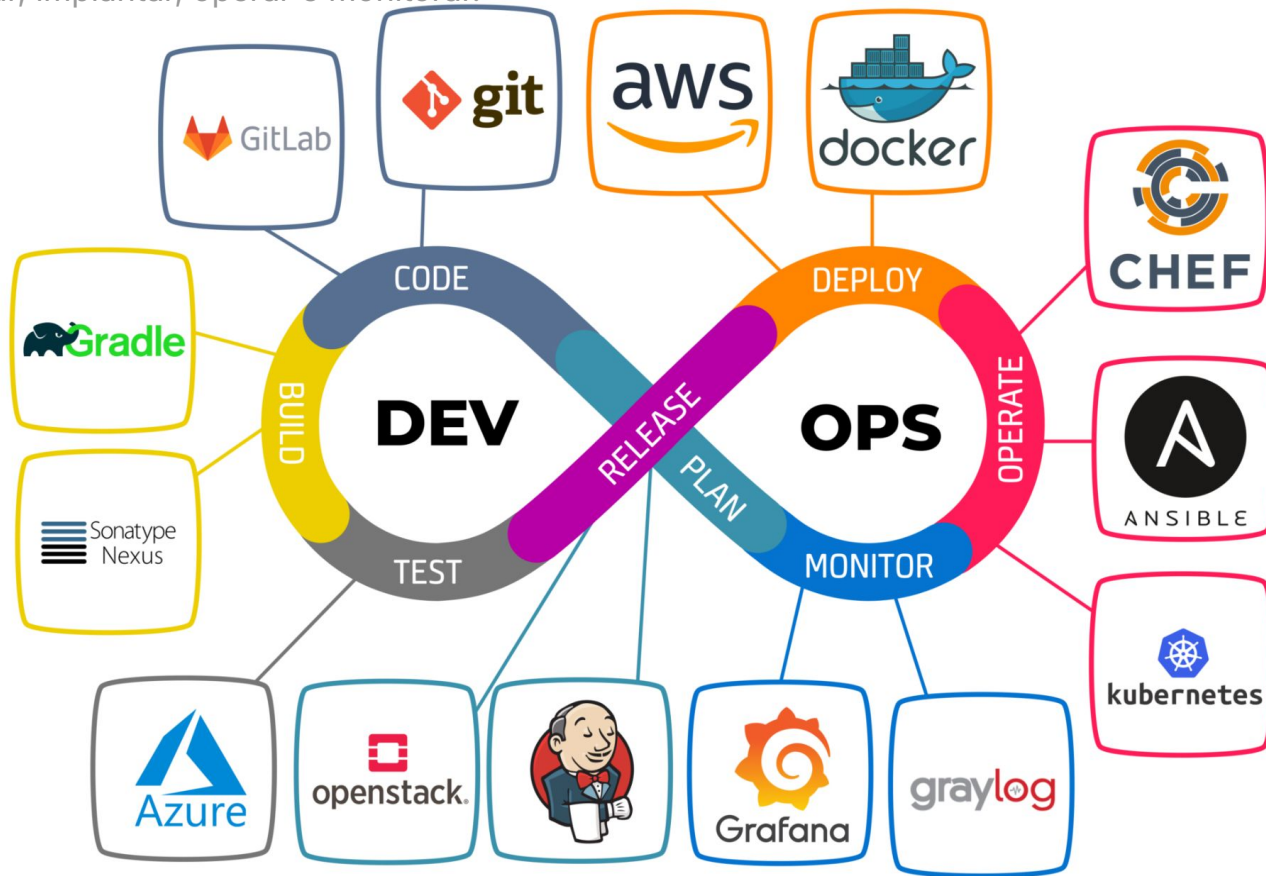


RUP e Agile podem coexistir

É possível ter governança inspirada em RUP com execução ágil e enxuta nas equipes.



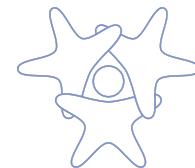
Integra desenvolvimento e operações em um fluxo contínuo de planejar, codificar, construir, testar, liberar, implantar, operar e monitorar.





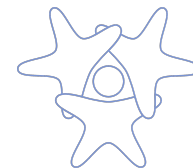
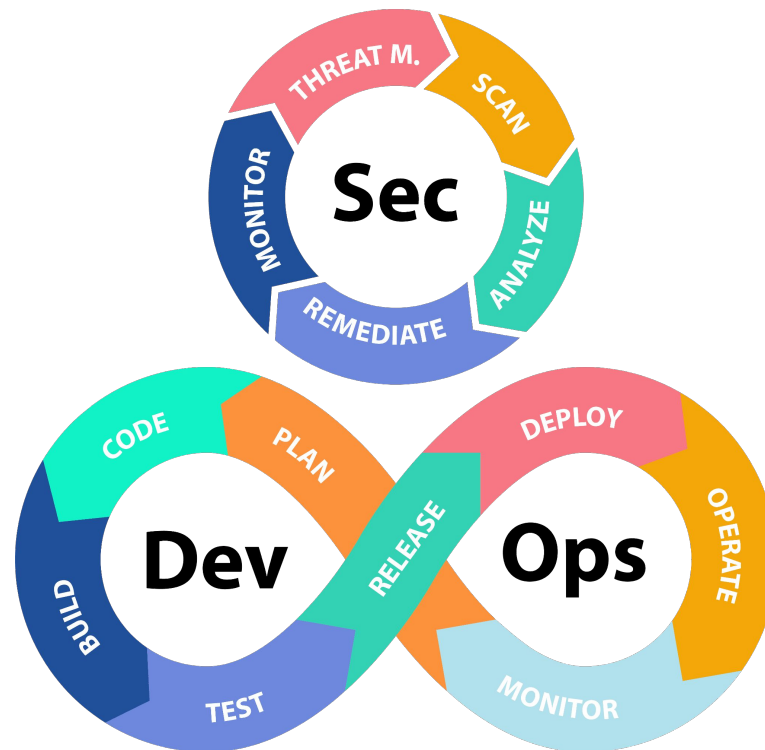
Observabilidade

Logs, métricas e traces ajudam a entender comportamento real do sistema em produção.



Segurança no ciclo

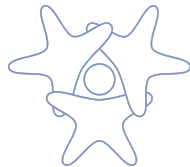
DevSecOps traz práticas de segurança para o fluxo contínuo, não apenas para o final.





Finalmente!

Não existe processo perfeito; existe processo coerente com contexto, risco, equipe, produto e organização.





Disciplina: Engenharia de Software e Gestão de Times Ágeis

RUP e Desenvolvimento de Software

Prof. Pedro Rodolfo Gomes de Souza
prgsouza3@gmail.com

